

Time Series with Konrad: episode 3

ARMA models and cousins - ARIMA, SARIMA, SARIMAX, oh my



KONRAD BANACHEWICZ

JAN 23, 2026

Until the 1980s, ARIMA models were the workhorses of time series analysis. For many practitioners, “doing time series” meant finding the right ARIMA model for the data. While newer approaches exist today, ARIMA and its extensions remain an essential baseline: they are fast, interpretable, and surprisingly effective.

In this episode, we’ll build these models step by step—from the simplest autoregressive processes to models that handle trend, seasonality, and external information. The goal is not mathematical rigor, but practical understanding: what these models are, why they work, and when to use them.

 *All code lives in the companion notebook:*

<https://github.com/tng-konrad/time-series-with-Konrad/blob/main/m03-arima-family.ipynb>

Linear models

At their core, ARIMA-family models describe a time series as a *linear combination* of:

- its own past values, and/or
- past random shocks (errors).

In plain English: today’s value depends on yesterday’s value and yesterday’s surprise.

More formally, these models assume that the data is generated by combining past values and past noise terms in a linear way. The noise is white :

- it has mean zero
- the variance is constant
- the series is uncorrelated over time

This setup gives us a unified framework for modeling a wide range of time series behaviors using relatively simple tools.

AR processes

The simplest example of a linear time series model is an autoregressive process of order p : in this model, today's value is expressed as a weighted sum of the last values plus random noise.

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \dots + \phi_p X_{t-p} + \epsilon_t$$

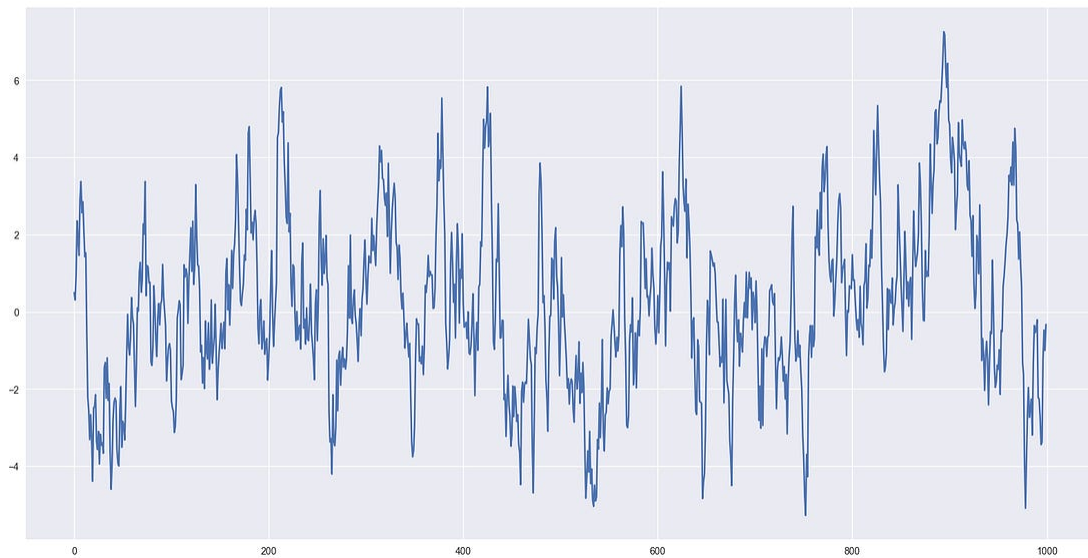
Let's unpack the formula:

- we forecast the variable of interest using a weighted sum of past values - direct extension of multiple linear regression
- autoregressive = we are regressing the series on itself
- the single exponential smoothing method (reminder: [here](#)) can be viewed as a special case of an $AR(1)$ model
- easy to estimate parameters
- fast to forecast

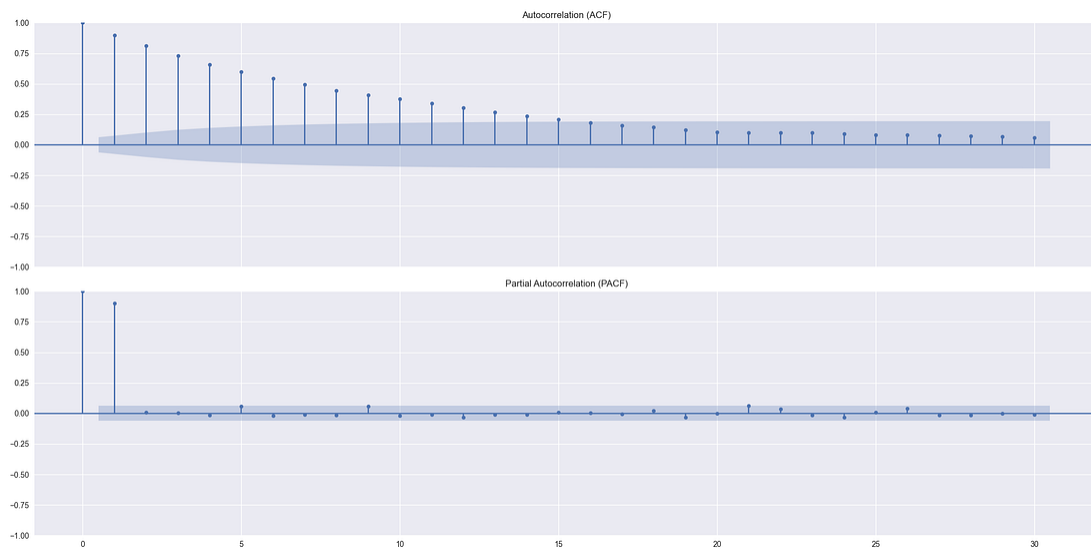
How can we identify an autoregressive process? Supposed we simulate our data from:

$$X_t = 0.9X_{t-1} + \epsilon_t$$

How would we recognize that this is an AR(1) process?



A common and practical approach is to inspect the autocorrelation function (ACF), and the partial autocorrelation function (PACF) - *check out [Episode 1](#) for a refresher.*



For an $AR(p)$ process:

- ACF is strong until lag p , trailing off afterwards

- PACF cuts off sharply after lag p .

This pattern gives us a quick diagnostic for choosing p .

MA processes

An autoregressive model uses past values of the series, moving average is based on past errors:

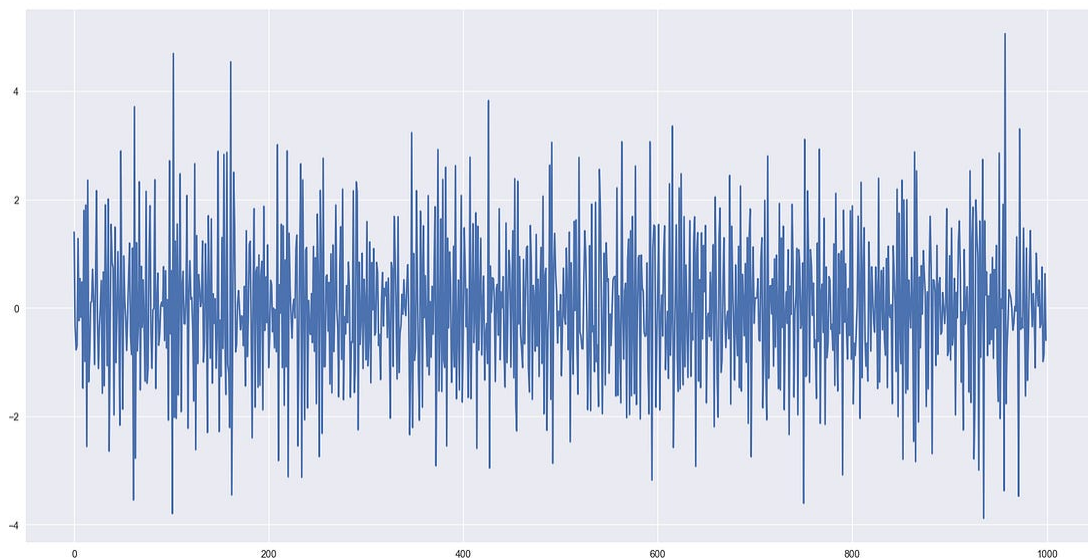
$$X_t = \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2} + \dots + \theta_q\epsilon_{t-q}$$

What does this mean?

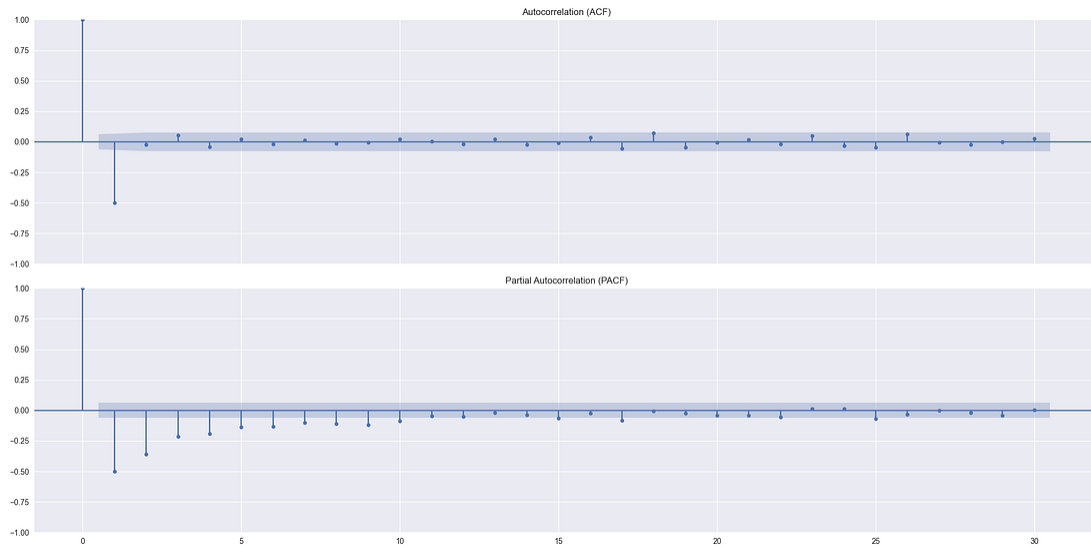
- Today's value depends on today's shock plus a weighted sum of past shocks.
- Persistence comes from past surprises (*as opposed to AR(p), where it comes from past values*).

Let's repeat the identification exercise from before: we simulate according to

$$X_t = \epsilon_t + 0.9\epsilon_{t-1}$$



What do ACF / PACF look like?



For an $MA(q)$ process:

- ACF cuts off sharply after lag q .
- PACF decays gradually.

To summarize: **AR and MA models show opposite patterns in the ACF/PACF, which helps us distinguish between them.**

ARMA

Many real-world series show both types of dependence: persistence from past values, **and** lingering effects from past shocks. We have an autoregressive component and a moving average one, so it is quite natural to combine those two types of dynamics into a single model: an $ARMA(p,q)$ series satisfies the relationship:

$$X_t = \phi_1 X_{t-1} + \dots + \phi_p X_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t$$

where our predictors on the right hand side include both lagged values of the series and lagged errors, p is the order the autoregressive part

and q is the order of the moving average component.

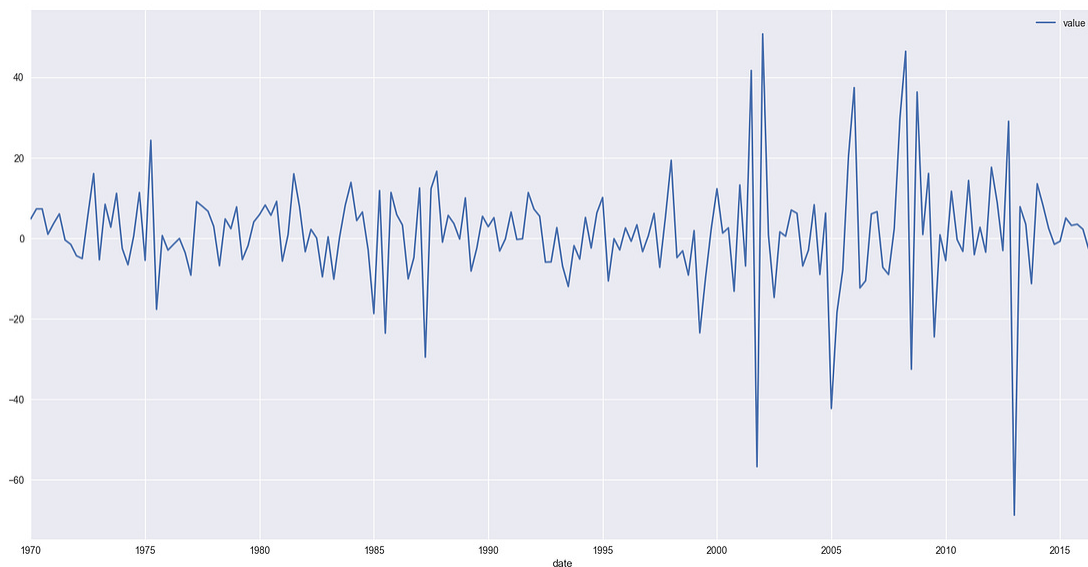
Key takeaways:

- ARMA models combine the strengths of AR and MA models.
- They are appropriate when the series is **stationary** (= no long-term trend or changing variance).

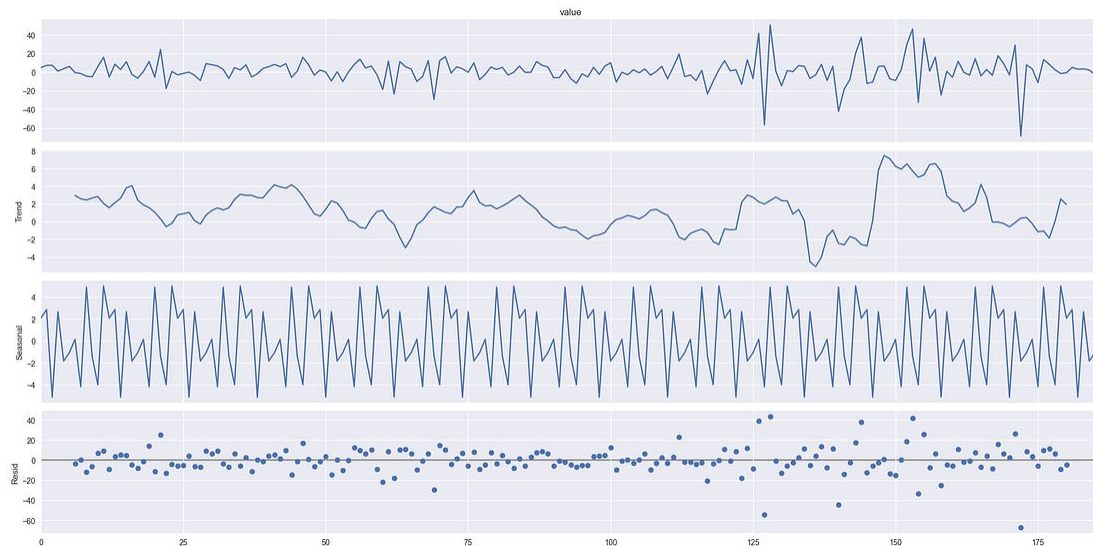
The proper analysis of stationarity involves discussing characteristic polynomials, which is above the math level I promised at the beginning of this series. In practice, in order to determine the order of an ARMA model we rely on:

- visual inspection,
- unit root tests (like the Augmented Dickey–Fuller test),
- and model diagnostics.

Let's put our newly introduced framework to the test and actually predict something. We look we will have a look quarterly changes in U.S. aggregate savings from FRED:



We start with a quick EDA: first, a seasonal decomposition

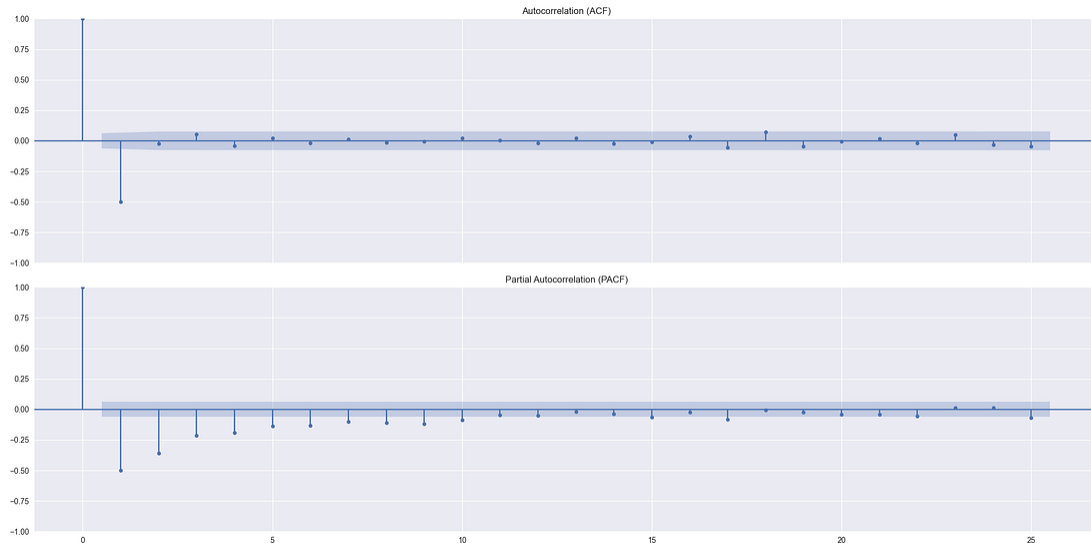


There is *a* seasonal component, but it is very small in magnitude; trend does not seem significant either. We suspect the process is stationary and move to testing this formally:

ADF Statistic: -18.705760

p-value: 0.000000

No red flags stopping us at this stage (*reminder: this means we don't see an overall linear trend during the sample period*) - we can proceed to model the series as $\text{ARMA}(p,q)$. What about the values of p and q ?



At this stage, we could make an educated guess at fitting an ARMA model - $(1,0)$ seems like a reasonable first pass. We can do better though: quickly try multiple models (for different pairs of (p,q)) and select the one with the lowest value of the information criterion.

We choose AIC: *Akaike Information Criterion*. It's a way to compare models that balances two goals: fitting the data well and keeping the model simple. Lower AIC values indicate better models - those that strike a better tradeoff between accuracy and complexity, which usually leads to better forecasts.

Performing stepwise search to minimize aic

ARIMA(2,0,2)(0,0,0)[0] : AIC=1417.652, Time=0.03 s

...

ARIMA(2,0,1)(0,0,0)[0] intercept : AIC=1417.096, Time=0.06 s

This automates a (somewhat tedious) task of establishing the right order for an ARMA model



In terms of forecast quality this model captures the general dynamics and it can be useful as a baseline - but overall it is clearly nothing to write home about:

1. there is a (predictable) lag: forecasts react one step after changes in the data - typical behavior for an autoregressive model
2. the model struggles with the range of values - this is a consequence of the constant variance assumption

This makes ARMA a good baseline, but not always a great final model. Can we do better?



Beyond ARMA

ARMA works well as long as the series is stationary. Real life time series are usually not : they have trends, they drift over time - and they exhibit seasonal effects. Enter ARIMA.

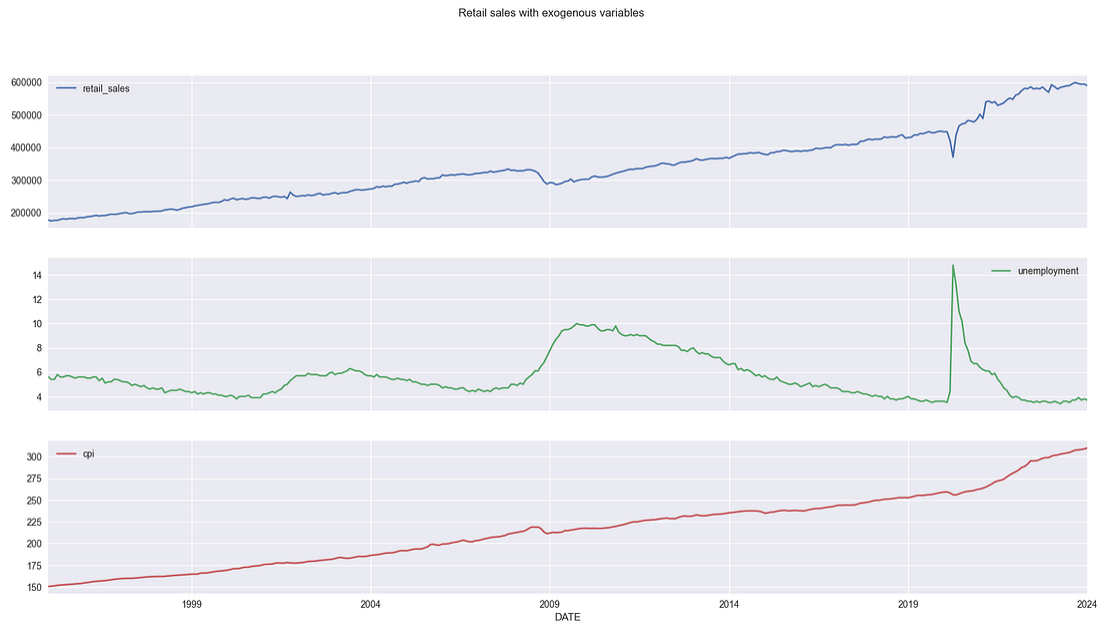
ARIMA

An ARIMA(p,d,q) process is defined as a series whose d -th difference is an ARMA(p,q) process.

Unpacking the (slightly cryptic :-) formulation:

- a series becomes a stationary ARMA(p,q) after differencing it d times
- differencing is a discrete version of taking a derivative: polynomial of order d becomes a constant after taking d th derivative $\Rightarrow$ a series stationary around a linear trend becomes stationary in first differences, quadratic trend - second differences etc)

Let's have a look at the retail sales data:



Applying Dickey-Fuller test confirms that the series is not stationary:

ADF Statistic: -0.22
p-value: 0.94

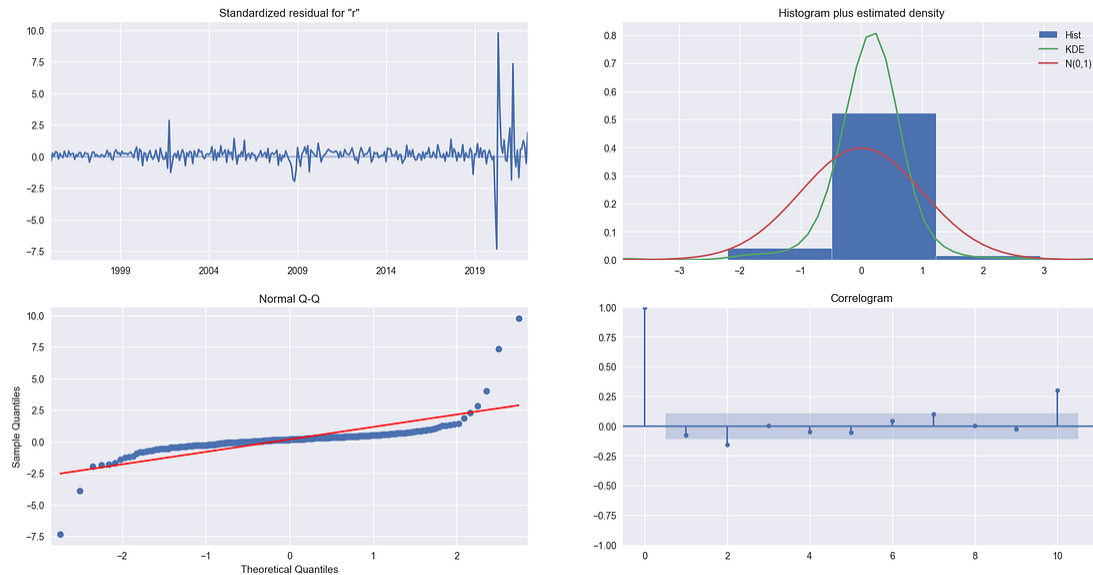
What order would work? We can find the best parameter combination automatically: a random walk with drift seems to be a winner.

Best model: ARIMA(0,1,0)(0,0,0)[0] intercept

Fit the model - how does it work?:

{'MAE': 24178.58, 'RMSE': 25608.27}

Diagnostics:



We captured the trend and the fit somewhat, but diagnostics show remaining seasonal structure. To capture repeating patterns, we extend it to SARIMA.

SARIMA

The “I” in ARIMA corresponds to the “integrated” component, which is a formal way of saying we are incorporating the trend into our setup starting with a stationary ARMA(p,q) process. Following a similar logic, we can incorporate the seasonal component and allow it to follow the same type of dynamic: a seasonal ARIMA (SARIMA) model can be denoted as $ARIMA(p,d,q)(P,D,Q)$, where:

- m refers to the number of periods in each season
- (lowercase) p,d,q refer to the definition of the ARIMA part
- (uppercase) P,D,Q refer to the autoregressive, differencing, and moving average terms for the seasonal part of the ARIMA model.

The last part is not entirely intuitive (*at least it wasn't to me when I learned it*). A good way to think about is that we want to identify components of the seasonal part the way we would for the series itself:

- We isolate the seasonal component, e.g. through seasonal decomposition
- To establish the value of D we test the seasonal component - is it stationary? is the differenced version stationary?
- The seasonal autoregressive value P can be established by inspecting the PACF of the seasonal component
- The seasonal moving average value Q can be established by inspect the ACF of the seasonal component

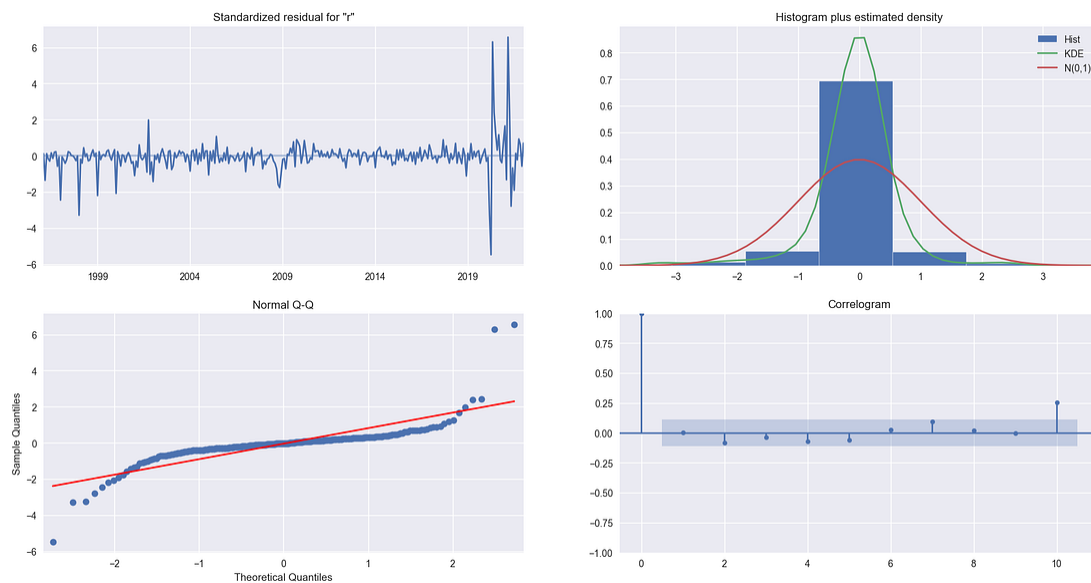
Select an optimal model based on AIC:

Best model: ARIMA(0,1,2)(0,1,2)[12]

Fit a model, the error we get:

{'MAE': 11643.2, 'RMSE': 14543.32}

Diagnostics:



There is clear improvement:

- **Error drops substantially** compared to ARIMA.

- Diagnostics improve.
- Seasonality is now explicitly modeled.

Finally, when we have external information about the future, we can move to SARIMAX.

SARIMAX

Sometimes, we know something about the future:

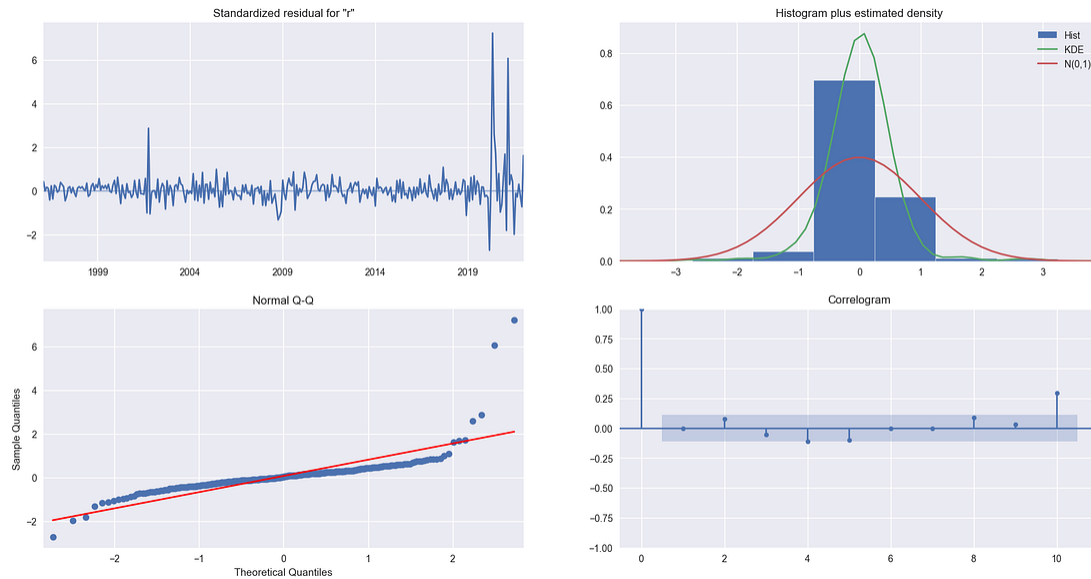
- economic forecasts,
- planned interventions,
- known calendar effects,
- or other time series that influence our target.

In those cases, we can include **exogenous variables**, giving us **SARIMAX**. Conceptually: we keep the same ARIMA/SARIMA structure, but add external predictors to improve forecasts.

Adding an explanatory variable to SARIMA fitted in the previous step gives us an error:

```
{'MAE': 17124.34, 'RMSE': 19863.89}
```

Diagnostics:



In this example, adding exogenous variables gives mixed results: better than raw ARIMA, but inferior to SARIMA. Takeaway message: more complexity does not always mean better forecasts.

Closing time

ARIMA and its extensions form a powerful toolkit for understanding and forecasting time series. Even AD 2026 – the era of machine learning, deep learning, and GenAI – these models remain valuable: they work well and they force us to think carefully about structure, trend, seasonality, and noise.

If you can fit a solid ARIMA or SARIMA model, you’ve already learned to “see” time series in a deeper way. Once you understand these foundations, more advanced methods become far easier to learn and evaluate.

Quick recap of ARIMA models:

Strengths

- Fast and interpretable.
- Excellent baselines.
- Strong for short-term forecasting in stable systems.

Limitations

- Require stationarity (or careful differencing).
- Assume linear relationships.
- Assume constant variance.
- Struggle with sudden regime changes and complex nonlinear dynamics.

United States of Banan is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ **Subscribed**



